RecoverML: Exact Artifact Time Travel for ML Pipelines under Declared Capability Loss

Abdul Hannaan
Qatar University
Doha, Qatar
aa2007325@qu.edu.qa

Abstract

Machine learning experiments leave many versions of data and models, and an old result may depend on a processing step that is no longer available. RecoverML chooses which files to keep for a defined set of possible losses and checks that recovered files match their original hashes. Its exact planners find and certify the smallest store under the stated physical file model. In a 56-case decision-rule study with the same robust storage limit, RecoverML restores all 3,344 requests while the tested HYPPO rank and frequency components restore 2,240 and 2,291 and the best tested HELIX rule setting restores 1,940. A later five-candidate storage choice uses exact deltas and joint compression and reduces storage against the earlier flat optimum in 244 of 444 measurable conditions, with 200 ties and a largest reduction of 89.84%. Against local search with the same formats it wins 20 and ties 424, with two Letter conditions unmeasured. The median saving against that control is zero and an equal-format exact reference matches the certified minimum. At the same optimal stored files, RecoverML is faster than the tested native DVC integration in all 864 census configurations while a direct executor is near parity. A final-file-API study recovers 32,256 of 32,256 requests exactly and measures higher throughput with four process replicas in all 12 fixed cases. The results show gains in recovery coverage, storage and command execution for the measured histories.

Keywords

historical machine learning recovery, data provenance, artifact retention, materialization, capability loss, reproducibility

1 Introduction

Researchers change data, features and model settings as an ML experiment develops, so one project can leave many related datasets, fitted transformers, models and predictions. HELIX and HYPPO use shared work to reduce repeated execution, and collaborative workload systems also organize these files for later reuse [1, 7, 18].

Older versions remain useful when someone needs to inspect an error or recover the output behind a reported result. In EyeAI, Li et al. recorded more than 70 datasets, 130 executions and 142 trained models, and their records helped trace poor diagnosis performance to an earlier optic nerve extraction step [10]. That case motivates keeping access to old results, though it does not measure the storage saving from our method.

For example, an image pipeline may turn raw images into features and then use those features to train a model and make predictions. If the old feature extractor disappears, keeping only the images will not recover the earlier features. Storing those features may be enough when training and prediction can still run, but if those steps may also disappear then the old models or predictions may need to stay. The choice depends on which historical outputs matter and which processing steps may fail.

DVC records dependencies and can freeze stages so they are not rerun [3], while Kishu restores earlier notebook states from checkpoints [9]. An old model can also require compatible software to load [15]. RecoverML addresses the file selection question: given a recorded history and a stated set of possible processing losses, which files must stay so every protected output can be recovered exactly?

The recovery contract names the protected targets, mandatory source files and groups of processing steps that may become unavailable together. For every allowed loss, each target must already be stored or be recoverable from stored files and surviving steps. The planner counts each distinct physical file once along with the required metadata, and hashes identify the exact bytes expected at recovery. The guarantee assumes complete recorded dependencies, intact stored files and a supported replay environment.

RecoverML uses exact optimization for the general recorded history and a minimum cut route when the graph passes its eligibility checks. Minimum cut is an established method [12]; here a matching integer flow and cut certify the selected store. A separate checker verifies the certificate and the executor checks recovered file hashes. When all requested targets are already stored, direct export returns the verified bytes without loading a model or starting numerical libraries.

We measure storage, planning and recovery separately against strong local search, published decision rules, manually chosen frontiers and native DVC [4]. In 36 histories under shared preprocessing loss, RecoverML uses less storage than local search in nine and ties in 27, reaching a 48.74% reduction in the largest case. All 18 histories that store an explicit concatenation file tie, and the manual best frontier matches every exact minimum.

In a native DVC comparison, both methods store the same optimal files. RecoverML has a lower median elapsed time in all 36 histories, with median paired DVC to RecoverML ratios of 2.19 for the 18 histories needing recomputation and 9.91 for the 18 whose targets are already stored. Each history has three paired repetitions, and the timing covers command startup and hash checking but excludes setup and planning. Earlier development revisions remain in the artifact and are not mixed into this final comparison.

The later storage study can keep full files, exact prefix and suffix deltas or a jointly compressed archive, and it chooses the smallest complete store from five candidates. It saves bytes against the earlier flat optimum in 244 of 444 measurable conditions and against local search with the same formats in 20, with 424 local ties. Two

Letter conditions remain unmeasured. A broader native DVC census compares recovery at the same optimal stored files in 864 configurations and includes a direct executor control [4].

In a separate study, 28 histories and two loss bounds give 56 cases. At the same robust byte limit and with one recovery executor, RecoverML restores all 3,344 requests per policy while the tested HYPPO components and HELIX rule settings recover fewer. These are comparisons of their adapted file selection rules rather than complete native system deployments [4].

In the larger Letter experiment, selective retention saves 3.6647 decimal GB of canonical bytes under preprocessing loss, or 290.61 MB with measured gzip. Under broader training and prediction loss it saves no bytes, strong robust heuristics match its storage choice and loading snapshots is faster in the measured recovery run.

Novelty and contributions. RecoverML brings together defined recovery requirements, checked file selection and exact recovery of old results. It uses established provenance and optimization methods and makes three systems contributions:

(1) **A clear recovery contract.** Protected file identities, mandatory roots and grouped processing losses define what must remain recoverable, while the storage objective counts distinct physical files and required metadata.
(2) **A checked system.** General exact formulations and a guarded minimum cut route choose files, certificate checks verify the choices and the executor checks recovered hashes. Direct export handles targets that are already stored.
(3) **Measured choices and costs.** Public data and retinal image histories show how loss grouping and capture boundaries affect storage, and later storage and native DVC studies measure encoding and recovery time with local and manual controls.

2 Related Work

2.1 Materialization and reuse across ML experiments

HELIX decides which intermediate results to store and whether to load or recompute them in a later iteration, using a flow formulation for reuse and heuristics for storage [18]. That work is a direct precedent for RecoverML, which fixes the old outputs that must remain recoverable and finds the smallest store that protects them against the specified processing losses.

Derakhshan et al. organize shared workloads in an Experiment Graph and select files using their size, computation cost and expected reuse [1]. Their later work groups shared computation in a DAG and chooses stored results under a byte budget to reduce future execution time [2]. RecoverML also uses a shared history, but its decision is based on complete recovery of specified old outputs after stated losses.

HYPPO records alternative producers in a directed hypergraph and uses equivalent results to find cheaper execution plans [7], while its later demonstration supports interactive exploration of shared histories [8]. RecoverML adds explicit recovery requirements and checks whether the selected files meet them. Our experiments run pinned HYPPO selection components and a HELIX rule adapter through a shared executor, so they compare these file choices under one byte limit rather than the complete native systems. We also test robust greedy and local search methods designed for our recovery requirement.

2.2 Scientific provenance and historical state recovery

Li et al. link datasets, features, executions and assets across scientific workflow versions, and EyeAI uses those records to investigate an earlier feature extraction problem [10]. RecoverML asks which files from such a history can be omitted while protected old outputs remain recoverable after specified producer losses. Our retinal experiment uses public images inspired by this case; it does not use the private EyeAI clinical workflow.

Kishu records notebook variable dependencies and restores earlier states with incremental checkpoints and checkout operations [9]. RecoverML protects selected immutable files after stated producer losses rather than the full interactive notebook state.

DVC records dependencies and reruns stages, while frozen stages can preserve chosen intermediate outputs without rerunning their producers [3]. A person can choose the same optimal file frontier in DVC, as our native controls confirm. RecoverML automates that choice and checks it against the recovery requirements; the separate latency experiment compares two authored integrations using the same paid stored files.

2.3 Replay and checkpoint tradeoffs

CHEX uses execution history and ordered checkpoints to share work across program versions [11]. RecoverML instead chooses persistent files so old outputs remain available after specified producers fail. Checkmate uses mixed integer optimization to trade neural training computation for activation memory [6]; RecoverML uses established optimization methods for a different file recovery objective.

ExoFlow separates execution from recovery and uses task annotations for recovery with exactly-once workflow semantics [19]. RecoverML checks the bytes of immutable output files in a supported replay environment, and its storage and recovery costs are measured separately because keeping fewer files can make later recovery slower. RecoverML also has a single-host atomic publication path for process crashes; it does not coordinate distributed side effects.

2.4 Optimization foundations and the remaining distinction

Implicit hitting set methods already combine a candidate choice with an oracle and learned constraints [13]. Minimum cost source location studies paid sources under connectivity requirements [14], and vertex splitting represents vertex costs in a flow network [12]. RecoverML builds on these methods rather than claiming them as new algorithms.

RecoverML applies those methods to recorded ML files and checks that its chosen store protects every target under every allowed loss. For eligible DAGs it uses a flow and cut certificate, while the general route handles other supported histories. In the

shared loss study, local search misses a smaller store in some cases, and the manual frontier and concatenation controls identify histories with simpler optimal choices.

3 Methodology

RecoverML records an experiment and its dependencies, then uses the recovery requirements to choose which files to keep. Figure 1 follows that choice through certificate checking, storage and later recovery. The executor uses only the selected files and their manifest; the full capture archive remains available for research checks but cannot supply missing files during recovery.

3.1 Captured history and physical file identities

A logical node records an operation, its inputs and parameters, the output hash and size, and whether the operation can run again. The environment record keeps package versions and implementation hashes for supported operators. Arrays and supported models use recorded serialization, so exact recovery means the same bytes rather than a model with similar accuracy.

Several logical nodes may point to one physical file, so the planner charges that file only once. Let B contain the captured physical blobs and let w_b be the nonnegative size of blob b. The map $\beta(p)$ gives the physical output of producer p, and $D(p)$ lists its physical inputs. More than one producer may supply the same blob, but a producer needs all its inputs. A nonreplayable node remains in the history without providing a recovery path.

Mandatory roots $R \subseteq B$ must stay and the protected targets are $T \subseteq B$, which may include models, probability arrays or older features needed for investigation. Every compared method uses the same manifest and roots. During recovery it cannot read an omitted file from the full capture archive, infer bytes from a hash or replace an old target with a new model.

3.2 Recovery contract and byte objective

The capability map places replayable producers in loss groups, and producers that depend on the same entry point may share a group. Ungrouped producers remain available. For the group set G and loss bound k, the allowed losses are

$$\mathcal{F}_k = \{F \subseteq G : |F| \le k\}. \tag{1}$$

The requirement covers processing loss while stored files and mandatory roots remain intact. A changed environment or corrupt stored file causes a refusal when safe recovery cannot be proved.

For a retained set S, recovery starts with $A_0 = S$ and adds the outputs of surviving producers when all their inputs are available:

$$A_{i+1} = A_i \cup \{\beta(p) : p \text{ survives } F,\ D(p) \subseteq A_i\}. \tag{2}$$

The process stops at the least fixed point $\mathrm{cl}(S, F)$. Starting from stored files stops an unsupported cycle from counting as a recovery path. The complete file selection problem is

$$\begin{aligned} \min_{S \subseteq B} \quad & m + \sum_{b \in S} w_b \\ \text{subject to} \quad & R \subseteq S, \\ & T \subseteq \mathrm{cl}(S, F) \quad \forall F \in \mathcal{F}_k, \end{aligned} \tag{3}$$

Here m is the size of the fixed manifest and serialized contract. Reported storage includes roots and metadata once. If a separate byte limit is too small, complete recovery is infeasible; maximizing partial recovery then becomes a different objective.

3.3 General exact formulations

The scenario formulation uses one set of binary file selection variables across all allowed losses, then records which files and producers are available in each loss case. A producer can run only after its inputs, so rank constraints stop an unsupported cycle from appearing to recover itself. The solver minimizes the file cost in Equation 3, and a grounded recovery check then tests its chosen store. The number of scenarios grows with the groups and loss bound, so the reference implementation refuses grids beyond its stated limit.

The second formulation starts with a storage master that has one selection variable x_b per physical blob. An oracle looks for an allowed loss F and a closed set C that contains the stored files but misses a target. Closed means that every surviving producer whose inputs are in C also has its output there. Such a witness shows that the store fails under F, so every valid store must satisfy

$$\sum_{b \in B \setminus C} x_b \ge 1. \tag{4}$$

The master adds this constraint and tries another cheapest store. Once the master proves its minimum and the oracle finds no failing loss, the selected store is both feasible and optimal within the recorded model. Timeouts and invalid solutions remain unknown rather than certified. This master and oracle design follows established hitting set and abduction methods [13].

Before accepting a solver result, RecoverML checks binary values, the objective, roots and learned constraints. It first tries the DAG route below; otherwise it uses the scenario formulation for at most 64 allowed scenarios and the master and oracle route above that number. This threshold is an engineering choice, so solver status and later checks still decide whether a result is certified.

3.4 Guarded DAG specialization and its certificate

The faster route applies only when the physical dependency graph is a DAG after mandatory roots are treated as available and every optional blob has at most one replayable producer. The guard rejects cycles and multiple producers of the same physical output, even when two logical producers wrote identical bytes.

For this subclass, universal recovery when $k \ge 1$ is equivalent to recovery with all grouped producers disabled together:

$$\bigcap_{F \in \mathcal{F}_k} \mathrm{cl}(S, F) = \mathrm{cl}(S, G). \tag{5}$$

This equality is a way to reason about a fixed store; it does not add a simultaneous loss of all groups to the runtime contract. In topological order, a stored blob is available, an unstored blob with

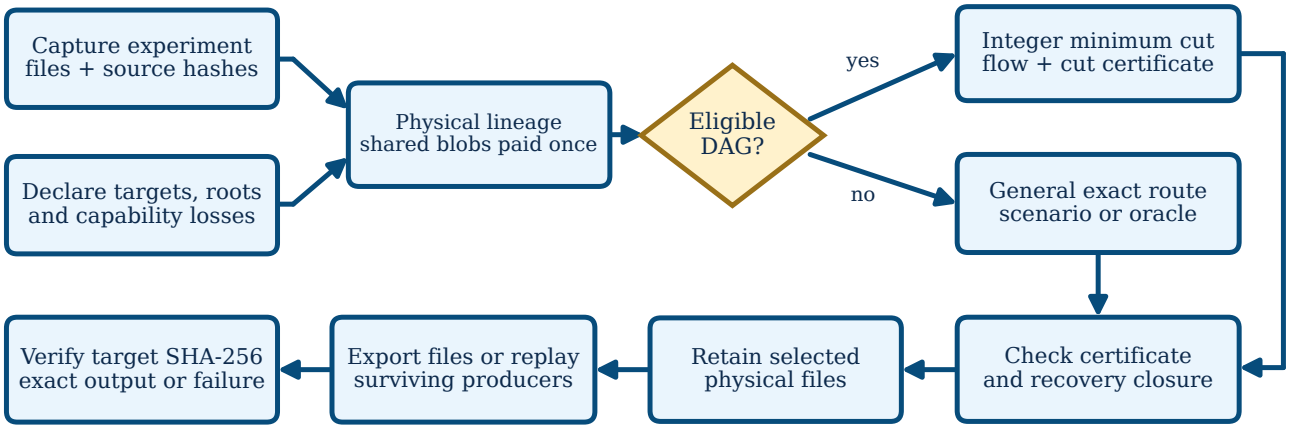


Figure 1: RecoverML capture, file selection and recovery.

a producer that may fail is not available under every allowed loss, and an ungrouped producer can always supply its output when all inputs are available. Disabling all grouped producers gives the same universal rule. When $k = 0$, no producer is disabled.

After this reduction, an unstored blob without a surviving producer is a failure source. Failure can pass through dependencies, and storing a blob stops it there, so protecting targets becomes a weighted vertex cut between failure sources and targets. Each optional blob becomes an input and output vertex joined by an edge of capacity w_b, while dependency and terminal edges receive capacity

$$M = 1 + \sum_{b \in B \setminus R} w_b. \tag{6}$$

Keeping every optional blob costs less than cutting an M edge. This is the standard vertex splitting construction [12]; roots and fixed metadata are added to the selected file cost.

NetworkX finds an integer maximum flow and a cut. The checker verifies edge capacities and flow conservation, then removes the chosen stored vertices and checks that no failure source can reach a target. The weight of the stored optional blobs must equal the feasible flow value: the cut gives an upper bound and the flow gives a matching lower bound. A separate grounded recovery check confirms that the store protects the targets.

This reduction does not apply to every AND/OR history. Two producers in different groups may each recover one target after a single group loss, although disabling both would prevent recovery; an unsupported physical cycle also breaks the cut argument. The guard therefore sends these histories to a general exact route.

3.5 Components and recovery execution

An optional decomposition removes mandatory roots from the live dependency graph and solves target bearing components separately. RecoverML combines their selected files and pays for global roots and metadata once; solver projections are not saved as capture manifests. Shared loss labels are safe here because each component is checked against every allowed loss relevant to it. There is also a direct rule for one optional target whose producers depend only on roots: the target need not be stored if an ungrouped producer exists or if the number of distinct producer groups exceeds k.

For an actual recovery request, the executor finds a grounded path from stored files through surviving producers and follows dependency order. Each request has its own memo and trace, and every loaded or recovered file must match its recorded hash. Missing or corrupt files and source mismatches produce explicit errors. The general restorer checks its supported runtime before loading values and never uses a producer declared unavailable.

When every requested target is already stored, direct export checks its size and SHA-256 identity and writes the output without loading a model. If a target is absent the executor uses ordinary replay, while a corrupt stored target causes an error. Exporting the same bytes does not mean an incompatible environment can run the model.

3.6 Publication and concurrent recovery

The durable publisher writes selected blobs, the manifest and contract to a sibling directory, flushes files and POSIX directories, then atomically renames it to the final path on the same filesystem. One publisher owns each destination and readers use only the final path: a process stopped before rename exposes no store, while one stopped after rename leaves a complete store. Published files are read-only; each request owns its restorer, memo, trace and output directory, and success requires exact target hashes. The legacy staging writer remains separate from this guarantee.

4 Experimental Setup

The experiments ask which files remain necessary under different loss requirements and whether other selection methods choose the

same stores. We then measure planning and actual recovery separately. Table 1 lists the fixed studies; one captured history can appear under several contracts or policies. Protocols and source snapshots are saved with the artifact [4].

4.1 Histories and declared processing changes

The shared loss grid uses Wine and Breast Cancer from scikit-learn [16]. A stratified split assigns 75% to training and 25% to testing under seeds 20261016, 20261017 and 20261018. Two disjoint feature views are standardized from training data only, and logistic regression versions cover regularization from 10^{-3} to 10^{3}. One capture boundary lets downstream steps consume both views directly while the other also stores their concatenated matrix.

Each history is tested first with separate preprocessing groups and then with one group covering both standardization steps, always with $k = 1$ and the same captured bytes. The shared loss follow-up came after a favorable pilot, so the complete grid retains small histories and explicit concatenation controls. It describes these authored histories rather than the frequency of outages in production.

The scientific matrix adapts Iris feature union, Digits PCA, Diabetes polynomial features and Digits forests. Two seeds and three versions give eight captures, tested under preprocessing and broad loss with $k \in \{0, 1, 2\}$. Its five budgets pay fixed roots and metadata plus 0, 10, 25, 50 or 100% of the optional target snapshot bytes, so they are set without using a later optimum. Eight methods produce 1,920 method and budget rows.

The retinal study uses all 20 STARE fundus images and Hoover vessel annotations [5]. Images become 64 × 64 RGB, with 15 training images, five test images and 6,000 sampled training pixels. Three seeds, two model families and three history lengths give 18 captures. An aligned extractor replaces a legacy three-pixel feature shift, producing different feature bytes. Targets include models and predictions and sometimes investigation features; the contracts cover intact processing, loss of the old extractor or loss of training and prediction. Model settings also change across versions, and the overlapping histories are not literal nested prefixes despite that wording in the original protocol.

The large study uses all 20,000 UCI Letter observations with 16 features and 26 classes [17]. Seed 20261015 gives a stratified split of 15,000 training and 5,000 test observations, while training only MinMax scaling and chi-squared selection keep 12 features. Sixteen trained forests combine 64/128/192/256 trees, two feature sampling settings and two model seeds. The saved files contain no padding or duplicated payload added to reach a storage target.

4.2 Comparators and budget fairness

The exact comparators solve the same full recovery problem. Robust greedy adds the largest grounded coverage gain per byte, while our stronger local search also removes unnecessary files from a full store and tries one-file exchanges followed by pruning. It allows at most two exchange passes. Target snapshots store the protected outputs directly, and a manual frontier control selects the smallest feasible frontier using knowledge of each workflow.

The HYPPO rank and frequency methods use a pinned released component and HELIX uses a documented rule adapter. They share the executor and byte accounting, though their selection rules target reuse. In the cloud grid they receive the exact minimum total byte budget while robust heuristics and snapshots can use the full candidate allowance for complete store comparisons. The scientific matrix gives every method the same fixed budget at each point, so the two studies answer different comparison questions.

The separate 56-case matrix gives RecoverML, both HYPPO functions and six HELIX settings the same robust byte limit. HYPPO uses its pinned released component through a physical file adapter. HELIX uses a documented streaming rule reimplementation at 50, 200 and 800 MiB/s with both recorded tie orders. A HELIX control with the larger snapshot budget is reported separately. Every policy uses the same grounded recovery executor, so these results compare file selection decisions [4].

The native DVC study uses DVC 3.67.1 and gives it the certified optimal retained payloads without charging selection time. One downstream batch stage avoids a separate Python startup for every model, and correctly retained target cache objects can let DVC skip that stage. Before each trial both methods have the same paid physical payloads and no unpaid downstream cache objects, with distinct-inode byte checks and hard links used for common retained files. DVC configuration is recorded separately from the common payload objective. Both authored workers can use the exact file-export helper.

The later encoded storage study retains the previous flat optimum as a candidate, adds exact prefix and suffix deltas with their bases and headers charged, and measures a joint archive of retained canonical payloads. The final selector computes the flat exact, delta exact, flat local, delta local and separately compressed choices and keeps the smallest complete store. We compare it with the previous flat and delta implementation, a joint archive of the flat optimum, and local search with all offered formats. The first three-candidate version lost three conditions to the strengthened local control; adding both local plans was a disclosed correction after that result. Every row in the later table uses the final five-candidate selector, and the original losses remain saved in the artifact [4]. The exact reference only certifies the declared flat or delta inventories, not a global optimum over every jointly compressed retained set.

The later storage catalogue has 216 primary, 48 scientific transfer, 72 three-seed cloud and 108 retinal conditions, plus two Letter conditions whose complete paired storage values are unavailable. These conditions share captures and were chosen for this project, so win fractions describe the catalogue rather than an estimated production probability. All stored byte totals include metadata, mandatory roots and delta bases. Saved file and export checks confirm exact identity but are not new full-grid model replay requests. The joint archive is decoded into memory and uses a temporary retained store for replay; its compression time and peak memory were not benchmarked.

The broader native DVC census has 72 captures from six datasets, four recipes and three history lengths. Three loss contracts, two request sizes and command line or warm API access make 864 configurations, each timed in three paired randomized-order repetitions. The methods receive the same certified optimal retained payloads. The elapsed call includes requested-file inventory and hash verification but excludes capture, planning and

Table 1: Workloads and comparison design.

Study	Data and model family	Captures; versions	Main comparison or control
Shared-loss study	Wine and Breast Cancer; logistic regression	36; 4/16/64	Three seeds, two capture boundaries, two failure groupings, nine policies
Native recovery	Fresh captures of the same recipe grid	36; 4/16/64	RecoverML versus native batched DVC at identical paid payloads; three paired repetitions
Published rules	Public tabular data; original HYPPO components and HELIX rule adapter	28; varied	Two loss bounds give 56 cases; 3,344 requests per policy at the common robust byte limit
Encoded storage	Six public datasets plus scientific, cloud and retinal transfers	446 conditions	Previous flat and delta choices, flat joint archive and equal-format local search; two Letter cases unmeasured
Recovery census	Six public datasets and four deterministic recipes	72 captures	Three loss contracts, two request sizes and two interfaces; 864 paired configurations at equal paid payloads
Scientific matrix	Iris, Digits and Diabetes; four recipes	8; 3 each	Two seeds, two loss scopes, three bounds, five budgets, eight policies
Retinal histories	20 STARE images; logistic regression and forest	18; 2/8/16	Two target modes, three availability contracts, five retention policies
Large payload	UCI Letter; trained random forests	1; 16	Preprocessing versus broad loss and canonical versus compressed representation
Final runtime scaling	Breast Cancer linear and Digits PCA-neighbor histories	6; 1/4/16	Root replay or retained target export; 1/2/4/8 threads or 1/2/4 process replicas; three repeats

setup, and OS caches are not flushed. A separately coded direct executor supplies a control for the cost of DVC orchestration [4].

4.3 Machines, repetitions and recorded outcomes

The shared loss study runs one fixed seed block on each of three Linux GitHub runners exposing four virtual CPUs on AMD EPYC 7763 hosts. The native study uses runners exposing four CPUs with EPYC 9V74 for one block and EPYC 7763 for two. Python is 3.12.15 with NumPy 2.3.5 and SciPy 1.17.0 and scikit-learn 1.8.0, and numerical execution is limited to one BLAS thread. These are separate runner blocks of the same constructed workload design rather than multi node scale-out.

Each cloud planner has three timed calls per case and the earlier 48-condition replanning study has five. General solver calls use a 30 s limit and a requested relative MIP gap of zero, with 120 s allowed in Letter. The polynomial flow route does not enforce a hard interruption deadline. A native recovery comparison has three paired trials with deterministically randomized method order. Its primary elapsed time includes cold subprocess and CLI startup and final common SHA-256 verification, while setup and planning are excluded and worker timings and CPU are logged separately. OS page caches are not flushed. The final native revision follows earlier exploratory corrections and is not pooled with them.

Every actual request records whether its expected target identities were recovered, whether a deficient store was correctly blocked and whether an unexpected outcome occurred. When methods select identical stores they share explicitly attributed execution evidence, so method totals must not be summed as new physical requests. Ratios are computed within a history from method medians and then summarized across histories. These descriptive ratios and repeated requests do not establish population significance.

Table 2: Storage choices against strong local search.

Loss grouping	Wins	Ties	Losses	Max. reduction
Separate	6	30	0	0.1942%
Shared	9	27	0	48.7380%

The final file API runs on a four-vCPU AMD EPYC 9V74 Linux runner. Breast Cancer and Digits have 1, 4 and 16 versions; root-only replay and retained-target export use different measured byte totals. Each condition has three bursts of 128 requests, using 1/2/4/8 threads or 1/2/4 same-host process replicas, for 252 trials. Requests have private outputs; warmup and planning are excluded. Throughput covers submission through completion, service P99 covers execution, and response P99 also includes queue wait. The artifact retains every request, source hash, machine record and independent audit.

5 Results

5.1 Storage choices under shared preprocessing loss

The two exact formulations agree on all 72 history and contract minima, and 216 repeated planning calls include checked flow proofs. Against strong local search, RecoverML uses fewer bytes in six of 36 separate loss cases and ties in 30, with each win measuring 129 serialized bytes. Under shared loss it wins nine of 36 and ties 27, with no losses. Table 2 gives the counts and Figure 2 shows all shared loss cases. The median saving across the full grid is zero and the manual best frontier matches every exact minimum. In the figure W and B mean Wine and Breast Cancer, each version count has three seed points, and reductions use the full paid store size.

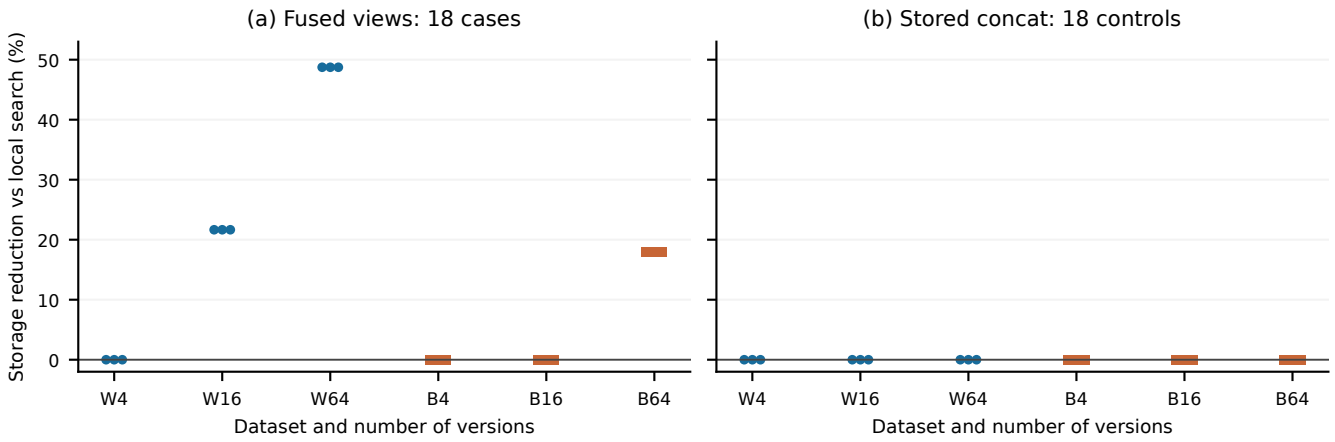


Figure 2: Storage reduction under shared preprocessing loss.

The nine shared loss wins occur in fused Wine histories with 16 or 64 versions and fused Breast Cancer histories with 64 versions across all three seeds. In the largest case RecoverML stores 138,618 bytes against 270,411 for local search, saving 131,793 bytes or 48.7380%. This decision gap is about 132 KB in absolute size; the separate Letter study measures the larger byte saving.

Both standardized views may be needed together to replace many models and predictions, which a one file exchange can miss. Recording their concatenation gives local search that joint choice as one file and closes the gap in all 18 explicit concatenation controls. The manual frontier also reaches every minimum.

5.2 Coverage and the meaning of a complete contract

At the cloud grid's exact minimum budget RecoverML is complete in 36/36 histories per loss grouping, compared with 18/36 for HYPPO rank and 21/36 for frequency and 15/36 for the HELIX rule. The robust full store controls remain feasible but can retain more bytes. Across physically unique cloud requests there are 16,110 exact results and 3,542 expected blocks out of 19,652, with zero unexpected outcomes. RecoverML's attributed requests are all exact, and the blocks belong to deficient adapter stores.

5.3 Published decision rules at one robust storage limit

Every policy in the 56-case matrix receives 3,344 historical requests and the same robust byte limit, and RecoverML restores all 3,344. Table 3 includes both HYPPO components and every tested HELIX speed and tie order. The HYPPO components restore 2,240 and 2,291 requests, while the best HELIX setting at the common limit restores 1,940. A separate HELIX control with the larger target snapshot budget restores 2,437 of 3,344 and is kept outside the common limit comparison [4]. The HYPPO rows use released ranking components through an adapter and HELIX uses the reimplemented rule, all with the same executor.

In the lossless subset every tested method restores all 320 no-outage requests. With one disabled group RecoverML restores

Table 3: Exact recovery with the same robust byte limit.

Policy	Exact	Requested
RecoverML	3,344	3,344
HYPPO rank	2,240	3,344
HYPPO frequency	2,291	3,344
HELIX 50 MiB/s, capture order	1,940	3,344
HELIX 50 MiB/s, reverse order	1,940	3,344
HELIX 200 MiB/s, capture order	1,681	3,344
HELIX 200 MiB/s, reverse order	1,681	3,344
HELIX 800 MiB/s, capture order	1,673	3,344
HELIX 800 MiB/s, reverse order	1,673	3,344

960/960, while HYPPO rank and frequency restore 729/960 and 818/960, and the HELIX 50 MiB/s capture setting restores 722/960. With two disabled groups the respective counts are 480/480, 278/480, 350/480 and 271/480. The distinction appears when the declared losses occur, although the rounded-import cases can also reflect the difference between protecting old targets and selecting files for future reuse. The original HELIX and HYPPO objectives differ from this preservation contract, and their complete deployed systems were not timed here. The 28 related histories and repeated requests do not estimate how often this difference would arise in independent production workflows.

The scientific matrix gives each method 240 conditions and 4,560 requests. Table 4 separates complete recovery of a condition from the number of individual requests recovered: RecoverML and local search each complete 128 conditions, while local search returns more individual targets across the budgets. RecoverML optimizes complete recovery when feasible, and at full budget local search matches all 48 exact storage minima. An expected blocked request can be the correct result when a fixed budget cannot meet the complete requirement.

5.4 Planning cost and component checks

In the new cloud grid the median paired local-to-RecoverML planning ratio is 11.92 under separate loss and 16.88 under shared loss,

Table 4: Recovery in the scientific fixed budget matrix.

Method	Complete conditions	Exact requests
RecoverML exact	128	1,734
Scenario exact	128	1,734
Robust greedy	112	2,221
Robust local	128	2,324
HYPPO rank	104	1,989
HYPPO frequency	100	1,856
HELIX rule	96	1,700
Target snapshots	112	1,862

and the scenario-to-RecoverML ratios are 2.28 and 1.73. These compare planning wall times rather than artifact recovery. The older 48-condition timing study retains the original minima and gives a 4.20 median ratio against local search, with RecoverML faster in 44 conditions and slower in four.

Figure 3 shows one point per condition and a black median bar, with five planning repetitions per method. In all 36 DAG eligible cases RecoverML is faster than local search and the median ratio is 6.27; among 12 fallback cases the median is 1.64 against local search and 0.93 against scenario exact. Ratios above one favor RecoverML, so the fallback is slower than scenario exact at its median, and four local comparisons are also slower.

A separate finite guard validation enumerates 2,275 DAG specifications with four artifacts and checks 18,200 closure comparisons for fixed stores and 127,400 minimum cut optima against exhaustive search, with no disagreement. These finite checks support the stated model and do not establish correctness for arbitrary unrecorded workflows.

The component study has 154 recorded rows including 110 rows under the same contract, and all 99 certified same contract objective comparisons agree. All 1,452 enumerated fault certificate checks and six indexed versus reference chain comparisons also agree. Decomposition is slower than the undecomposed route in six of eight settings from public recipes, so a correct split does not guarantee lower overhead. Changing logical identity is a change of contract rather than an optimization ablation for the same objective. The separate annotation study with 5,760 conditions changes selected sets in 298/4,608 cost profile comparisons and 312/2,880 access weight comparisons, which shows that the adapter outcomes depend on their declared annotations. These are structural and planner checks and not additional timed disk recoveries.

5.5 Native recovery at identical retained storage

The final native comparison completes 216 full history subprocess commands and 12,096 exact target hash checks. RecoverML has lower median elapsed time in all 36 histories. Figure 4 shows the three trial median for each history against an equality line and plots the paired ratios separately for histories that need replay and histories whose targets are stored. Times include startup and verification while setup and planning are excluded, and OS caches are not flushed.

For the 18 histories needing downstream computation the median paired DVC-to-RecoverML ratio is 2.1856, with ratios from 1.7215 to 2.5109. For the 18 histories whose targets are already retained it is 9.9067, with ratios from 8.9527 to 11.3402. The medians of the per history absolute times are 1.663 s for RecoverML and 4.146 s for DVC in the recomputation stratum, and 0.0633 s versus 0.6056 s in the retained target stratum. A ratio of these aggregate medians need not equal the median of paired ratios.

The direct export path avoids numerical startup when replay is unnecessary and both workers have access to the same helper. DVC still pays the orchestration cost of its native command and stage handling, so the large retained target ratio should be read as a property of these cold command integrations. An earlier revision with the corrected paid cache was mixed: RecoverML won all 18 recomputation cases but DVC won all 18 retained target cases. The export improvement was added after that result and evaluated in a fresh full grid. Earlier versions remain separate evidence and are not pooled with the final revision.

5.6 Complete encoded storage comparison

Table 5 compares the final five-candidate choice with four storage controls using total paid bytes. The denominator is 446; one Letter case has only the selected store and another lacks its original archive, leaving 444 measured pairs. Smaller, equal and larger count each comparison, while the 10% column uses all 446 conditions. The final choice includes the old plan and both local plans, so it cannot increase bytes against those candidates. Its earlier three-candidate form had 20 wins, 421 ties and three losses against the stronger equal-format local control; the two added local candidates correct those losses, and both versions remain recorded [4].

Against the previous flat optimum the largest reduction is 89.84%, from 1,457,715 to 148,050 bytes in `digits/pca_neighbors/v16/broad`. The largest gap against the equal-format local control is 70.17%, from 118,126 to 35,242 bytes in `shared/wine-s20261016-v64-concat0`. Most of that local gap already existed before joint compression, so it is not evidence of a new 70% compression gain. The median saving against local search is zero and only nine of 446 conditions reach 10% savings against that control. The inherited broad improvement test, which required at least 10% storage saving in 90% of histories per stratum, passes in zero of 72 strata.

The equal-format exact reference ties the measured flat and delta optima under their declared candidate inventories, and the manual best frontier ties all 36 minima in the earlier shared-loss study. An exact minimum cannot be strictly beaten under identical rules on the same byte objective. The joint archive changes those rules, while a global minimum over every possible jointly compressed retained set has not been established. The result is a systems gain in particular captured conditions, with no measured storage regression for the corrected portfolio, rather than strict dominance over every close comparator.

The independent release audit rebuilt the archive controls from saved canonical payloads and reproduced both local selectors. It reconciled 5,061 selected-file checks and 3,531 retained-target export checks, including repeated artifacts, and all 29 cloud jobs and the final audit passed. These checks are evidence for the saved

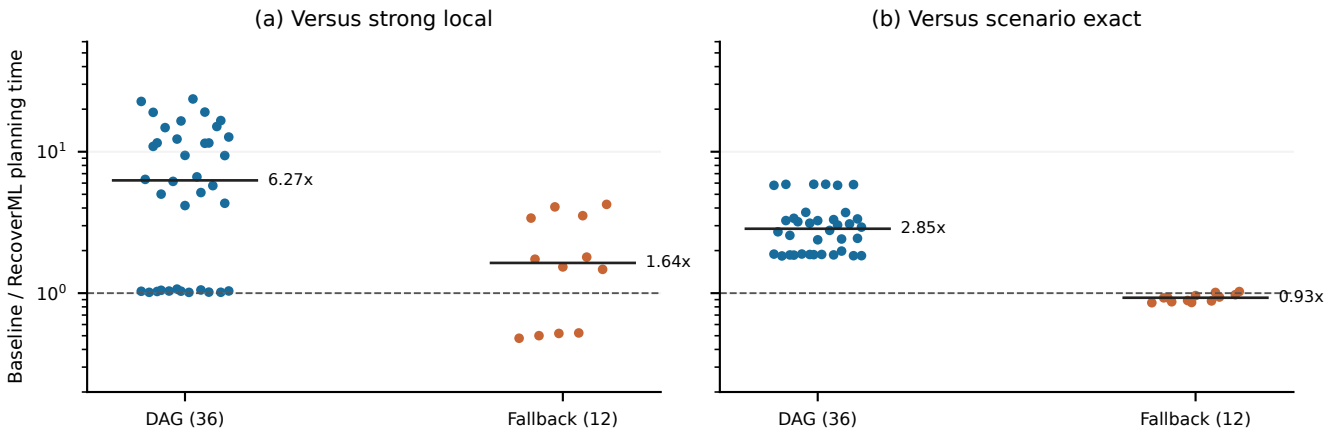


Figure 3: Planning time ratios across 48 conditions.

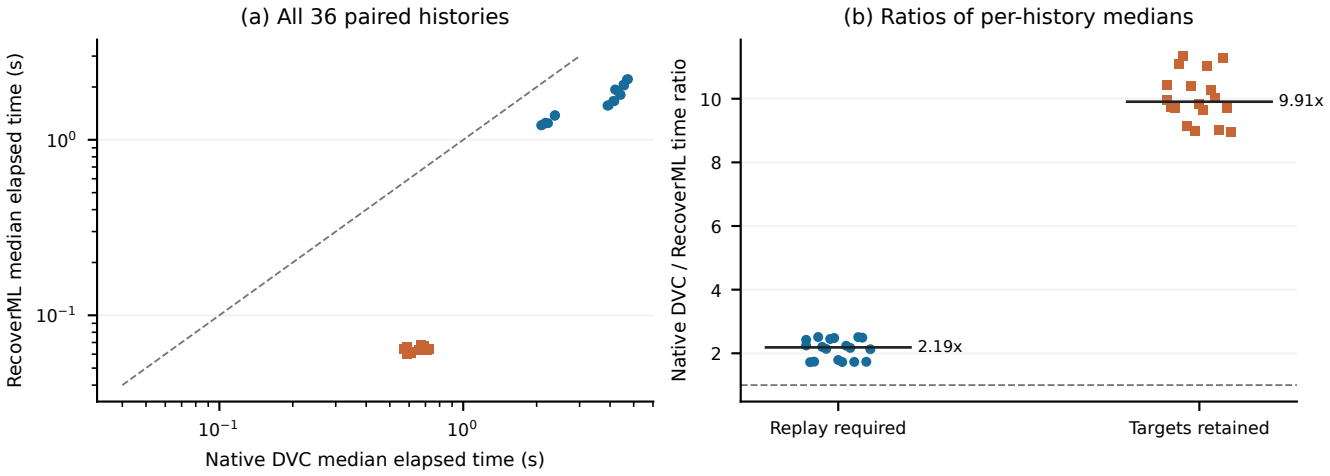


Figure 4: Native DVC and RecoverML recovery times.

Table 5: Storage outcomes for the five-candidate choice.

Comparator	Smaller	Equal	Larger	Unmeasured	Median saving	10%+
Previous flat optimum	244	200	0	2	0.094%	52
Previous flat or delta choice	214	230	0	2	0%	47
Joint archive of flat optimum	233	211	0	2	0.056%	5
Local search with all offered formats	20	424	0	2	0%	9

stores and file identities, not 8,592 new full-model replay requests. No full-catalogue compression time, random access latency or peak memory advantage is claimed [4].

5.7 Broader native DVC recovery census

In the later census RecoverML exceeds the prespecified 1.10 times elapsed-time threshold against the tested native DVC integration in all 864 configurations. The median paired DVC-to-RecoverML ratio is 1.6952 for the 432 command line configurations and 48.3212 for the 432 warm API configurations. All 7,776 trials restore requested files exactly and the ledger checks 54,540 requested-file hashes. Latest and all-version requests coincide when a history has one version, so these counts do not represent independent workloads [4].

The direct executor control narrows the interpretation. Its median elapsed time divided by RecoverML time is 0.9929, slightly favoring the direct control, and RecoverML reaches a 1.10 times improvement in only 21 of 864 direct comparisons while being at

least 1.10 times slower in 48. The native DVC difference therefore mainly measures orchestration in these integrations. Reusing DVC repository objects, other cache policies and remote recovery were not tested. This census supports a consistent advantage against the specified native path across the authored catalogue, but it does not establish faster model training or a production win probability.

5.8 Historical processing change and large payloads

The retinal study completes 10,920 exact requests across 540 method rows with no unexpected errors, and Figure 5 shows all 108 storage conditions. Colors mark the availability contract, circles and triangles mark logistic regression and forests, and small offsets separate the three seeds. Adding investigation features changes which files must be protected, while broad training and prediction loss can require target snapshots. Exact retention uses fewer bytes than target snapshots in 90 cases and ties in 18, improves on simple greedy in 12 and ties in 96, and matches strong local search and a simple frontier rule in all 108 cases. These related conditions reuse the same 20 public images.

The largest retinal snapshot saving is 12,316,663 bytes in an intact forest history whose targets include investigation features. In a separate native control the legacy module is physically absent before and after reproduction and all 32 protected targets are recovered exactly. A manually retained DVC frontier matches the common 2,523,881-byte objective. The main grid's declared restrictions and this physical removal control should not be described as the same failure experiment.

For Letter, preprocessing loss reduces the canonical store from 3,669,545,821 to 4,829,269 bytes, saving 3,664,716,552 bytes or 99.8684%, which exceeds 3.66 GB. With gzip the corresponding sizes are 291,175,536 and 568,375 bytes, saving 290,607,161. Figure 6 shows these decimal MB values on a logarithmic axis, with each RecoverML bar beside its target snapshot control. Strong robust heuristics match the preprocessing optimum and broad loss gives no canonical or gzip saving. Native pickle and native gzip controls have different byte identities and are recorded separately.

The actual Letter replay ledger contains 48 exact requests across three unique stores under all preprocessing groups disabled, a stronger stress than its $k = 1$ selection contract. The broad-loss entries are structural checks and are not additional actual runtime requests. In the descriptive preprocessing replay, mean frontier recovery is 6.65 s per version compared with 2.12 s for snapshot loading. The evaluation includes all 16 trained forests and their held-out accuracy ranges from 95.0% to 95.64%, but improving predictive accuracy is not the retention objective.

5.9 Concurrency, tail latency and process crashes

The final file API recovers all 32,256 requests exactly across 252 audited trials. Figure 7 compares each dataset, version count and mode with its one-worker result; points are medians of six fixed-case ratios and shading shows their range. The modes use different storage: at 16 versions, root-only and target-file stores cost 172,570 and 205,679 bytes for Breast Cancer and 975,357 and 3,117,275 for Digits.

Four process replicas increase throughput in all six fixed cases per mode: median ratios are 2.50 for replay and 1.97 for export. Eight threads reduce throughput in all 12 cases, with median ratios of 0.70 and 0.48. Median service-P99 ratios at four processes are 1.59 and 3.09; at eight threads they are 11.75 and 22.38. The saved raw requests expose each tail sample and its queue wait. The earlier prototype separately records 5,400 exact requests across 54 trials.

We also ran root-only replay on independent Linux runners, with one, two or four replicas each capturing the same declared 16-version Digits history. Across three synchronized rounds per configuration, all 2,688 requests matched their own runner's recorded target hashes. Median aggregate throughput was 35.2, 73.3 and 141.1 requests/s, or 2.08 and 4.00 times the one-runner rate. The four-runner rate was 80% of the sum of its individual rates. The runners used two CPU variants and produced two target hash sets, so these ratios describe heterogeneous provisioned capacity rather than equal-hardware speedup. Each replica used its own immutable store; no networked shared store was tested.

Process-crash tests kill a child just before or after publication: the final path is absent before rename and verifies completely after it. A separate 32-reader test recovers exact bytes into private outputs from one immutable store.

6 Discussion and Reproducibility

RecoverML first protects complete recovery and then minimizes stored bytes, while recovery time is measured separately. The manual frontier ties and faster snapshot loads show how the preferred choice depends on that objective. The repository provides source hashes, captured file identities, raw timings, solver records and analysis scripts [4]. Its script rebuilds the original figures from recorded inputs, and later storage and recovery ledgers are linked in the same artifact. Fresh runs write to a new output directory, and none of these studies required a GPU.

7 Limitations

The datasets are public, but the histories and loss cases were built for these experiments; related seeds, versions and requests do not estimate a production win rate. The favorable shared loss pilot informed a later full grid, and the five-candidate store was corrected after the stronger local control exposed three losses. The retinal study adapts public images rather than the private EyeAI workflow, and Letter's byte saving changes with compression. Annotation effort, deployment cost and how often capabilities disappear remain unmeasured.

Fresh capture and replay passed on three runners, but exact replay across arbitrary software changes was not tested. Six original nonforest scientific captures lack a closed global source guard after an interruption, although their snapshots, request guards and raw reconciliation remain. One original Letter archive container is unavailable; four recovered frontier files match their hashes in a separately identified replacement archive. The joint archive decodes into memory, so its compression time, random access cost and peak memory need measurement. Larger neural states, networked scale out, power-loss recovery and distributed crash consistency remain future work.

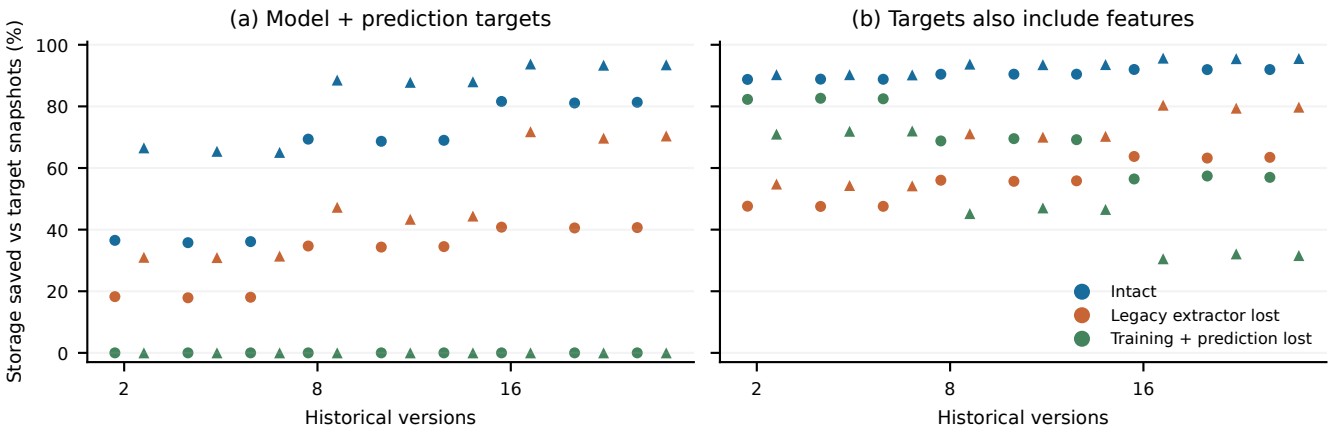


Figure 5: Retinal storage savings across 108 conditions.

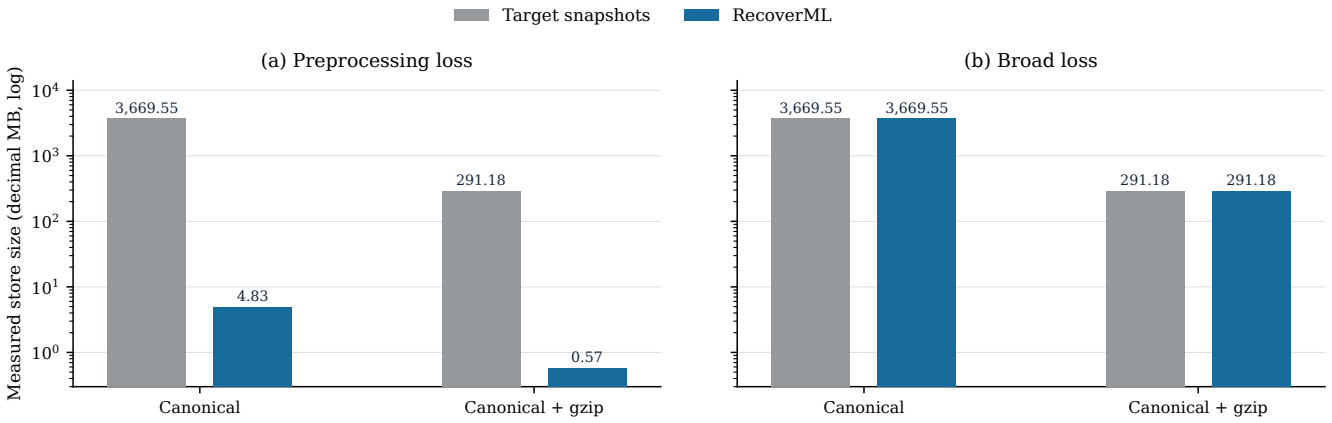


Figure 6: Letter store sizes under preprocessing and broad loss.

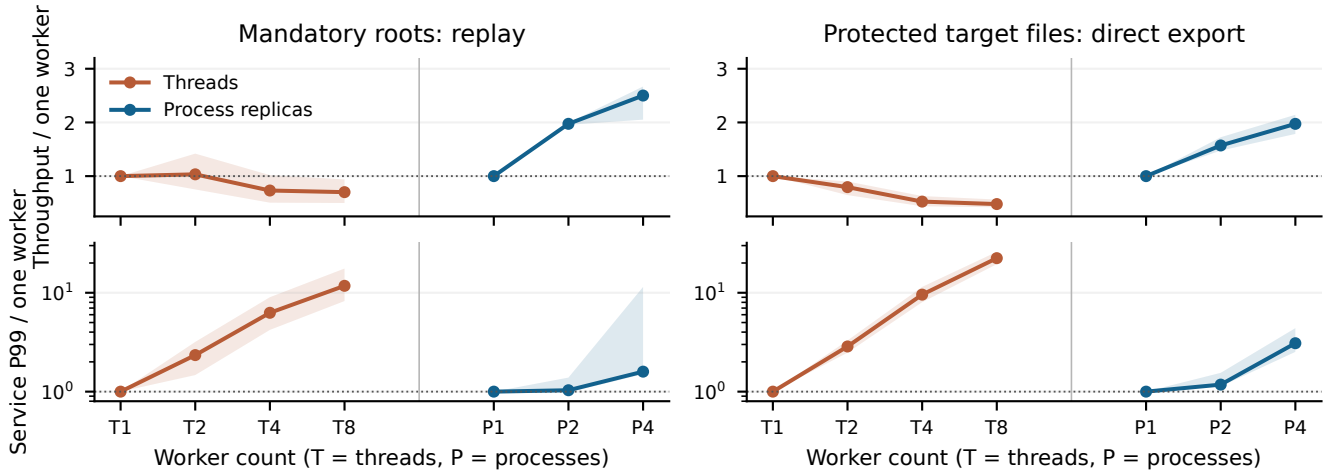


Figure 7: Final file API throughput and service P99 relative to one worker.

8 Conclusion

RecoverML records old ML files and chooses a checked store that keeps protected outputs recoverable after specified processing losses. It restores all 3,344 requests per policy in the 56-case decision-rule study at the same robust byte limit, where the tested HYPPO components and HELIX rule settings recover fewer. The five-candidate storage choice improves on the earlier flat optimum in 244 of 444 measurable cases and on equal-format local search in 20, with 424 local ties. The native DVC census also favors RecoverML at the same stored files, while a direct executor is near parity and manual or equal-format exact choices match the relevant storage minima. These results show where the system helps in the measured histories.

References

[1] Behrouz Derakhshan, Alireza Rezaei Mahdiraji, Ziawasch Abedjan, Tilmann Rabl, and Volker Markl. 2020. Optimizing Machine Learning Workloads in Collaborative Environments. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1701–1716. doi:10.1145/3318464.3389715

[2] Behrouz Derakhshan, Alireza Rezaei Mahdiraji, Zoi Kaoudi, Tilmann Rabl, and Volker Markl. 2022. Materialization and Reuse Optimizations for Production Data Science Pipelines. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1962–1976. doi:10.1145/3514221.3526186

[3] DVC contributors. 2026. *Data Version Control: Reproduction and Frozen Stages*. https://doc.dvc.org/command-reference/repro See also the freeze command reference. Accessed 17 October 2026; experiments use DVC 3.67.1.

[4] Abdul Hannaan. 2026. RecoverML Reproducibility Artifact. https://github.com/Hannaancode/RecoverML/tree/efae825be13ffe487b915a4580cdc16b4c5e95c6 Source and recorded experimental evidence at commit efae825, accessed 28 October 2026.

[5] Adam Hoover, Valentina Kouznetsova, and Michael Goldbaum. 2000. Locating Blood Vessels in Retinal Images by Piecewise Threshold Probing of a Matched Filter Response. *IEEE Transactions on Medical Imaging* 19, 3 (2000), 203–210. doi:10.1109/42.845178

[6] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. *Proceedings of Machine Learning and Systems* 2 (2020), 497–511. https://proceedings.mlsys.org/paper_files/paper/2020/file/0b816ae8f06f8dd3543dc3d9ef196cab-Paper.pdf

[7] Antonios Kontaxakis, Dimitris Sacharidis, Alkis Simitsis, Alberto Abelló, and Sergi Nadal. 2024. HYPPO: Using Equivalences to Optimize Pipelines in Exploratory Machine Learning. In *Proceedings of the IEEE International Conference on Data Engineering*. IEEE, 221–234. doi:10.1109/ICDE60146.2024.00024

[8] Antonios Kontaxakis, Dimitris Sacharidis, Alkis Simitsis, Alberto Abelló, and Sergi Nadal. 2025. Hyppo: Efficient Discovery and Execution of Data Science Pipelines in Collaborative Environments. In *Proceedings of the International Conference on Extending Database Technology*. OpenProceedings.org, 1134–1137. doi:10.48786/edbt.2025.107

[9] Zhaoheng Li, Supawit Chockchowwat, Ribhav Sahu, Areet Sheth, and Yongjoo Park. 2024. Kishu: Time-Traveling for Computational Notebooks. *Proceedings of the VLDB Endowment* 18, 4 (2024), 970–985. doi:10.14778/3717755.3717759

[10] Zhiwei Li, Carl Kesselman, Tran Huy Nguyen, Benjamin Yixing Xu, Kyle Bolo, and Kimberley Yu. 2025. From Data to Decision: Data-Centric Infrastructure for Reproducible ML in Collaborative eScience. arXiv:2506.16051. doi:10.48550/arXiv.2506.16051

[11] Naga Nithin Manne, Shilvi Satpati, Tanu Malik, Amitabha Bagchi, Ashish Gehani, and Amitabh Chaudhary. 2022. CHEX: Multiversion Replay with Ordered Checkpoints. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1297–1310. doi:10.14778/3514061.3514075

[12] MIT 6.854. 2016. *Lecture Notes: Advanced Flow Algorithms*. https://courses.csail.mit.edu/6.854/16/Notes/n9-advanced_flow_algorithms.html Accessed 17 October 2026.

[13] Paul Saikko, Johannes P. Wallner, and Matti Järvisalo. 2016. Implicit Hitting Set Algorithms for Reasoning Beyond NP. https://www.dbai.tuwien.ac.at/user/wallner/files/SaikkoWJ2016.pdf Author version with full proofs and extended related work.

[14] Mariko Sakashita, Kazuhisa Makino, and Satoru Fujishige. 2007. *Minimum Cost Source Location Problems with Flow Requirements*. Technical Report METR 2007–39. University of Tokyo. https://www.keisu.t.u-tokyo.ac.jp/data/2007/METR07-39.pdf

[15] scikit-learn developers. 2026. *Model Persistence*. https://scikit-learn.org/1.8/model_persistence.html scikit-learn 1.8 documentation. Accessed 17 October 2026.

[16] scikit-learn developers. 2026. *Toy Datasets*. https://scikit-learn.org/1.8/datasets/toy_dataset.html scikit-learn 1.8 documentation. Accessed 17 October 2026.

[17] David Slate. 1991. Letter Recognition. UCI Machine Learning Repository. doi:10.24432/C5ZP40 Dataset.

[18] Doris Xin, Stephen Macke, Litian Ma, Jialin Liu, Shuchen Song, and Aditya Parameswaran. 2018. HELIX: Holistic Optimization for Accelerating Iterative Machine Learning. *Proceedings of the VLDB Endowment* 12, 4 (2018), 446–460. doi:10.14778/3297753.3297763

[19] Siyuan Zhuang, Stephanie Wang, Eric Liang, Yi Cheng, and Ion Stoica. 2023. ExoFlow: A Universal Workflow System for Exactly-Once DAGs. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Boston, MA, USA, 269–286. USENIX paper.